UNIDAD 2 · CONECTORES A SGBD

Semana 3 · Transacciones, integridad y gestión de errores en JDBC

Teoría desarrollada para el alumnado · Java 21 · JDBC · MySQL 8.4 (InnoDB) · IntelliJ IDEA · Maven

Qué aprenderemos

Comprender por qué varias operaciones deben tratarse como una unidad; controlar autoCommit, commit() y rollback(); reconocer errores y restricciones; utilizar correctamente SQLException; y aplicar estas ideas a una transferencia segura entre cuentas.

1. ¿Por qué necesitamos transacciones?

Imagina dos cuentas: Ana tiene 500,00 € y Luis tiene 100,00 €. Queremos transferir 50,00 € de Ana a Luis. Debemos realizar dos actualizaciones SQL: restar 50 a Ana y sumar 50 a Luis. Si solo se ejecuta la primera, desaparecen 50 euros del sistema. Una transacción agrupa ambas operaciones para que se confirmen juntas o se deshagan juntas.

Momento	Cuenta Ana	Cuenta Luis	Total
Antes	500,00	100,00	600,00
Tras restar (temporal)	450,00	100,00	550,00
Tras sumar y confirmar	450,00	150,00	600,00
Si falla: rollback	500,00	100,00	600,00

Comprobar saldo → Restar → Sumar → COMMIT

Idea clave

Una transacción no consiste simplemente en escribir dos sentencias seguidas. Significa que, para tablas transaccionales como InnoDB, ambas se gestionan como una unidad: o terminan confirmadas o se revierten antes de confirmarse.

2. Las propiedades ACID

Propiedad	Significado	Aplicación al ejemplo
Atomicidad	Todo o nada.	No permitir que se reste sin sumar.
Consistencia	El estado debe respetar las reglas y restricciones.	El saldo y las relaciones siguen siendo válidos.
Aislamiento	Las transacciones concurrentes no deben interferir indebidamente.	Dos transferencias simultáneas no deben pisarse.
Durabilidad	Los cambios confirmados sobreviven a fallos dentro de las garantías del SGBD.	Una transferencia confirmada no desaparece tras reiniciar.

Matiz importante: ACID no sustituye a las validaciones de negocio. Por ejemplo, debemos comprobar el saldo disponible; MySQL no adivina que una cuenta no debe quedar en negativo si no establecemos esa regla.

3. Auto-commit: qué ocurre por defecto

En JDBC, las conexiones nuevas suelen iniciarse con autoCommit = true. Cada instrucción SQL se confirma individualmente cuando termina. Es cómodo para una inserción aislada, pero peligroso si varias sentencias dependen entre sí.

```
System.out.println(conexion.getAutoCommit()); // Normalmente true

// Si ambas operaciones se ejecutan así, cada UPDATE puede confirmarse
// por separado: no forman una única transacción de negocio.
```

Modo	Qué ocurre	Cuándo usarlo
autoCommit = true	Cada sentencia se confirma individualmente.	Operaciones independientes.
autoCommit = false	Se mantienen los cambios pendientes hasta commit/rollback.	Operaciones relacionadas que deben ser atómicas.

4. Los tres métodos esenciales de Connection

```
conexion.setAutoCommit(false);  // Control manual
// Ejecutar las operaciones SQL necesarias
conexion.commit();              // Confirmar todo
// o bien:
conexion.rollback();            // Deshacer lo no confirmado
```

setAutoCommit(false): desactiva la confirmación automática para esa conexión. commit(): confirma los cambios realizados en la transacción actual. rollback(): revierte los cambios pendientes de esa transacción. No puede deshacer una transacción que ya se confirmó.

setAutoCommit(false) → UPDATE 1 → UPDATE 2 → commit / rollback

5. Preparar la base de datos de demostración

Utilizaremos una base de datos de pruebas separada de la utilizada en otras semanas. Las sentencias siguientes se ejecutan una vez desde MySQL. Se usa ENGINE=InnoDB, porque necesitamos soporte de transacciones.

```
CREATE DATABASE IF NOT EXISTS jdbc_transacciones;
USE jdbc_transacciones;

CREATE TABLE IF NOT EXISTS cuentas (
    id INT PRIMARY KEY,
    titular VARCHAR(60) NOT NULL,
    saldo DECIMAL(10,2) NOT NULL,
    CONSTRAINT chk_saldo CHECK (saldo >= 0)
) ENGINE=InnoDB;

INSERT INTO cuentas (id, titular, saldo)
VALUES (1, 'Ana', 500.00), (2, 'Luis', 100.00);

SELECT id, titular, saldo FROM cuentas ORDER BY id;
```

Si ya habías ejecutado este script, no repitas el INSERT sin comprobar los registros, porque los identificadores 1 y 2 ya existirán. Para reiniciar el ejemplo puedes usar UPDATE cuentas SET saldo = CASE id WHEN 1 THEN 500 WHEN 2 THEN 100 END WHERE id IN (1,2);.

> Configuración necesaria
>
> El usuario JDBC debe tener permisos para acceder a jdbc_transacciones. Por ejemplo, un administrador de MySQL puede conceder GRANT SELECT, INSERT, UPDATE, DELETE ON jdbc_transacciones.* TO 'alumno'@'192.168.56.%';. Ajusta la subred a tu laboratorio.

6. Ejemplo A: transferencia confirmada

A continuación se muestra un programa completo. Primero crea una conexión, desactiva auto-commit, ejecuta dos UPDATE mediante PreparedStatement, comprueba que se haya modificado exactamente una cuenta en cada caso y confirma con commit().

```
import java.math.BigDecimal;
import java.sql.*;

public class TransferenciaOK {
    public static void main(String[] args) {
        String url = "jdbc:mysql://192.168.56.10:3306/jdbc_transacciones";
        String usuario = "alumno";
        String clave = "CAMBIAR_POR_CONTRASENA";
        BigDecimal importe = new BigDecimal("50.00");

        try (Connection c = DriverManager.getConnection(url, usuario, clave)) {
            c.setAutoCommit(false);
            try {
                String restar = "UPDATE cuentas SET saldo = saldo - ? "
                              + "WHERE id = ? AND saldo >= ?";
                String sumar = "UPDATE cuentas SET saldo = saldo + ? WHERE id = ?";
                try (PreparedStatement p1 = c.prepareStatement(restar);
                     PreparedStatement p2 = c.prepareStatement(sumar)) {
                    p1.setBigDecimal(1, importe);
                    p1.setInt(2, 1);
                    p1.setBigDecimal(3, importe);
                    if (p1.executeUpdate() != 1)
                        throw new SQLException("Origen inexistente o saldo insuficiente");

                    p2.setBigDecimal(1, importe);
                    p2.setInt(2, 2);
                    if (p2.executeUpdate() != 1)
                        throw new SQLException("Destino inexistente");
                }
                c.commit();
                System.out.println("Transferencia confirmada");
            } catch (SQLException e) {
                c.rollback();
                System.out.println("Transferencia cancelada: " + e.getMessage());
            }
        } catch (SQLException e) {
            System.out.println("Error de conexión o transacción: " + e.getMessage());
        }
    }
}
```

El ejemplo usa BigDecimal para representar importes. DECIMAL(10,2) en MySQL y BigDecimal en Java evitan las imprecisiones habituales de double en cálculos monetarios.

7. Analizar el código, línea por línea

Instrucción	Qué hace y por qué importa
setAutoCommit(false)	Evita confirmar el primer UPDATE antes de saber si el segundo funciona.
WHERE id = ? AND saldo >= ?	Solo descuenta si existe la cuenta y tiene fondos suficientes.
executeUpdate() != 1	Comprueba que la modificación afectó a exactamente una fila.
throw new SQLException(...)	Interrumpe la secuencia para entrar en el catch y revertir.
commit()	Confirma las dos modificaciones una vez verificadas.
rollback()	Deshace las modificaciones no confirmadas cuando falla algo.

Si la primera actualización afecta a cero filas, puede ser porque la cuenta no exista o porque no haya saldo suficiente. El programa no debe continuar con el ingreso. Si la segunda actualización falla, el primer descuento también se revierte.

8. Ejemplo B: provocar un error y comprobar el rollback

Para simular un fallo sin tocar el servidor ni borrar tablas, cambia el identificador del destinatario del ejemplo anterior por uno que no exista, por ejemplo 999. La segunda sentencia afectará a cero filas; el programa lanzará la excepción prevista y ejecutará rollback().

```
p2.setInt(2, 999);  // Cuenta de destino inexistente
if (p2.executeUpdate() != 1)
    throw new SQLException("Destino inexistente");
```

Situación	¿Se resta a Ana?	¿Se ingresa a Luis?	Estado final
Todo correcto	Sí, temporalmente	Sí	450 / 150 tras COMMIT
Destino 999 inexistente	Sí, temporalmente	No	500 / 100 tras ROLLBACK

Prueba observable

Después de cada ejecución consulta SELECT * FROM cuentas ORDER BY id;. Antes de repetir la demostración, restablece los saldos iniciales para que la comparación tenga sentido.

9. Gestión de SQLException: errores técnicos y de negocio

En JDBC, SQLException representa errores detectados durante operaciones de base de datos. Es una excepción comprobada: Java nos obliga a tratarla o declararla. No confundamos una consulta que devuelve cero filas con un error de ejecución.

```
catch (SQLException e) {
    System.err.println("No se pudo completar la operación");
    System.err.println("SQLState: " + e.getSQLState());
    System.err.println("Código del fabricante: " + e.getErrorCode());
    // En una aplicación real: registrar los detalles técnicos.
}
```

Caso	Tratamiento recomendado
Usuario solicita importe negativo	Validar en Java antes de acceder al SGBD.
Cuenta inexistente / saldo insuficiente	Informar de la condición; no confirmar la transferencia.
Conexión rechazada	Informar de que el servicio no está disponible y registrar el error.

Caso	Tratamiento recomendado
Clave duplicada / restricción CHECK	Capturar SQLException; describir el fallo sin revelar datos internos.
SELECT sin filas	Es un resultado válido; mostrar “No se encontraron registros”.

10. try-with-resources y cierre de recursos

La conexión, los PreparedStatement y los ResultSet implementan AutoCloseable. El bloque try (...) los cierra al salir, incluso si ocurre una excepción. El cierre no sustituye al control transaccional: hay que decidir explícitamente cuándo confirmar o revertir.

```
try (Connection c = DriverManager.getConnection(url, usuario, clave);
     PreparedStatement ps = c.prepareStatement("SELECT * FROM cuentas");
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        System.out.println(rs.getString("titular"));
    }
} // Se cierran rs, ps y c en orden inverso.
```

> Atención
>
> Si se utiliza un pool de conexiones en proyectos más avanzados, antes de devolver la conexión al pool hay que terminar toda transacción pendiente y restablecer sus opciones. Para las prácticas de esta semana, cada ejemplo utiliza su propia conexión y la cierra.

11. Una segunda herramienta: Savepoint

Un punto de guardado permite deshacer solo la parte de una transacción posterior a ese punto. Es útil cuando un proceso contiene operaciones opcionales. Se crea con setSavepoint() y se utiliza con rollback(savepoint). No reemplaza al commit final.

```
c.setAutoCommit(false);
Savepoint punto = c.setSavepoint("antes_de_operacion_opcional");
try {
    // Operación opcional que podría fallar
} catch (SQLException e) {
    c.rollback(punto);  // Vuelve al punto de guardado
}
c.commit(); // Confirma únicamente los cambios conservados
```

12. Errores habituales y límites de las transacciones

Error frecuente	Corrección
Olvidar setAutoCommit(false)	Desactivarlo antes de ejecutar la primera operación relacionada.
Hacer commit antes de comprobar todas las filas	Comprobar resultados y reglas antes de confirmar.
Usar rollback después de commit	Una confirmación completada ya no se puede revertir con ese rollback.
Suponer que CREATE TABLE se revierte siempre	En MySQL muchas instrucciones DDL hacen commit implícito.
Usar tablas MyISAM para la práctica	Utilizar InnoDB, que admite transacciones.
Confiar en close() para decidir el resultado	Realizar commit o rollback explícitos.

Algunas instrucciones de definición de estructuras (DDL, como CREATE TABLE) provocan confirmaciones implícitas en MySQL. Por eso, las prácticas transaccionales se centran en operaciones INSERT, UPDATE y DELETE sobre tablas InnoDB. Este detalle está documentado en el manual de MySQL 8.4.

13. Guía de diagnóstico paso a paso

¿Hay conexión? → ¿SQL válido? → ¿Filas esperadas? → ¿COMMIT?

Cuando una operación no da el resultado previsto, primero verifica el estado del servidor, después la consulta y sus parámetros, y después cuántas filas ha modificado. Si falla una operación de la transacción, revisa que se haya ejecutado rollback() y que no se hayan confirmado cambios parciales.

14. Actividad de razonamiento (sin soluciones)

Situación A: Una transferencia consta de dos UPDATE, pero el programador deja auto-commit activado. El primero funciona y el segundo falla. Explica el estado final y la razón.

Situación B: La cuenta de destino no existe. executeUpdate() devuelve 0. ¿Debe el programa ejecutar commit()? Justifica la respuesta.

Situación C: Dos alumnos ejecutan una transferencia al mismo tiempo. ¿Qué propiedad ACID está relacionada con esa concurrencia?

Situación D: La consulta SELECT devuelve un ResultSet vacío. ¿Hay que llamar a rollback() necesariamente? Explica por qué.

15. Resumen de conceptos

Concepto	Definición breve
Transacción	Grupo de operaciones confirmado o deshecho como unidad.
autoCommit	Modo de confirmación automática por sentencia.
commit()	Confirma la transacción actual.
rollback()	Revierte los cambios no confirmados.
Savepoint	Punto intermedio para una reversión parcial.
SQLException	Información de un fallo JDBC/SGBD.
BigDecimal	Tipo Java apropiado para cantidades monetarias exactas.

Fuentes técnicas de consulta: documentación oficial de MySQL 8.4 sobre autocommit, COMMIT y ROLLBACK, y documentación de Java sobre la interfaz Connection.

16. Consulta y refuerzo

Para profundizar: manual de MySQL 8.4 sobre transacciones InnoDB y documentación de Java sobre Connection.